ASSIGNMENT 1

Implementation of Breadth First Search (BFS) and Depth First Search (DFS).

```
//In the name of Allah
#include <bits/stdc++.h>
using namespace std;

// Implementation of Breadth First Search (BFS) and Depth First Search
(DFS).
/*
Sample graph with following inputs::
8 8
0 1
1 2
2 3
3 6
1 4
2 5 4 5 6 7

0-1-2-3-6
  | |   |
  4-5   7
*/
#include <bits/stdc++.h>
using namespace std;

void dfs(vector<int> g[], int vis[], int i){
    if(vis[i]) return;
    vis[i] = 1;
    cout << "Node "<<i<< " visited. \n";

    for(auto x:g[i] ){
        dfs(g,vis,x);
    }
}

void bfs(vector<int> g[], int vis[], vector <int> &q, int c, int i){
    if(i>=q.size()){
        return;
    }
    if(vis[q[i]]){
```

```
        bfs(g, vis, q, c+1, c);
        return;
    }

    vis[q[i]] = 1;
    cout << "Node "<< q[i]<< " visited \n";

    for(auto x: g[q[i]]){
        // cout << x<< " "<<vis[x]<<endl;

        if(vis[x]==0){
            q.push_back(x);
        }
    }

    // cout << c<<" : "<<q[c]<<endl;
    bfs(g, vis, q, c+1, c);
}

int main(){
    int n, e; cin>> n>> e;
    vector<int> grph[n];
    for(int i=0; i<e;i++){
        int a, b; cin>>a>>b;
        grph[a].push_back(b);
        grph[b].push_back(a);
    }

    // for(int i=0; i<n; i++){
    //     cout << i<<" : ";
    //     for(auto x: grph[i]){
    //         cout<< x<< " ";
    //     } cout << endl;
    // }

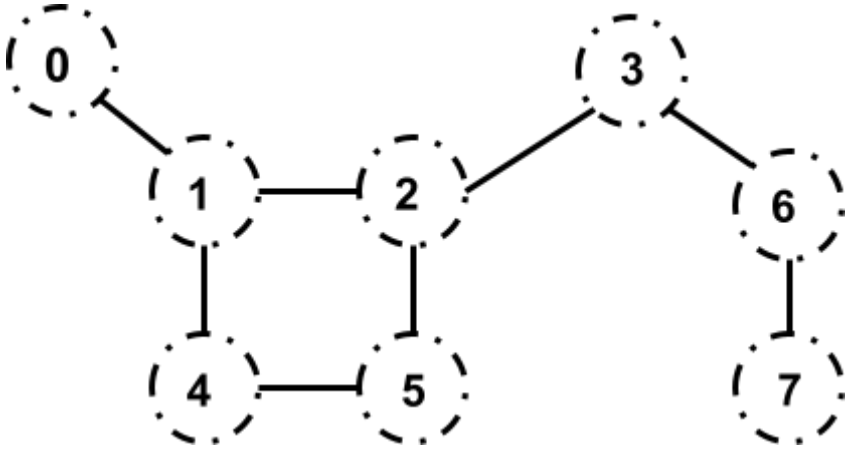
    int vis[n];
    memset(vis, 0, sizeof(vis));
    for(auto x:vis){
        cout << x<< " ";
    } cout<<endl;
// void dfs(vector<int> g[], int vis[], int i){
    cout << "DFS of the graph: \n";
    dfs(grph, vis, 0);
```

```
    memset(vis, 0, sizeof(vis));
    // for(auto x:vis){
    //     cout << x<< " ";
    // } cout<<endl;
    cout <<endl;

    vector<int> q;
    q.push_back(0);
// void bfs(vector<int> g[], int vis[], vector //<int> &q, int c, int
i){
    cout << "BFS of the graph: \n";
    bfs(grph, vis, q, 1, 0);

    return 0;
}
```

INPUT



OUTPUT

```
DFS of the graph:
Node 0 visited.
Node 1 visited.
Node 2 visited.
Node 3 visited.
Node 6 visited.
Node 7 visited.
Node 5 visited.
Node 4 visited.
```

```
BFS of the graph:
Node 0 visited
Node 1 visited
Node 2 visited
Node 4 visited
Node 3 visited
Node 5 visited
Node 6 visited
Node 7 visited
```